

Machine-Checked Translation Adequacy for Bhartṛhari’s *Vākyapadīya*: Commentary-Driven Verification of a Sanskrit Philosophical Text in Lean 4

Sharath Sathish

with an agentic formalization loop built on Claude (Anthropic)

<https://github.com/SharathSPHD/vakya-vallari>

July 2026

Abstract

Translations of dense philosophical Sanskrit are notoriously under-determined: a single *kārikā* admits multiple grammatically valid renderings whose philosophical commitments differ radically, and the commentarial tradition — not surface grammar — is what fixes the intended reading. We introduce *commentary-driven translation validation* (C-TV), to our knowledge the first application of an interactive theorem prover to translation adequacy in linguistics, and apply it to the complete *Brahmakāṇḍa* (Book I, 144 verse units) of Bhartṛhari’s *Vākyapadīya*. From an original English translation and analytical commentary of all 1,796 verses we derive, per verse, a *semantic contract*: sorted entities, identity, relation and predication claims, and explicit denials, every axiom citing the commentary verbatim — a mechanical anti-fabrication gate. A generator compiles contracts into Lean 4, where a decidable adequacy predicate proves that the accepted translation satisfies the contract (144 adequacy theorems) and that 350 plausible mistranslations provably fail — 153 contradicted by commentary denials, 197 unlicensed by the axioms — for 549 machine-checked theorems with zero **sorry**. The corpus was formalized by an agentic swarm gated by the validator and the Lean kernel; the gates caught real errors at every scale, from fabricated citations in degraded agent output to cross-contract ontological drift, adjudicated into a registry of 41 polysemous terms with 91 textually justified senses — including the *akṣara* pun that Bhartṛhari’s own commentators call the argument of the opening verse in miniature. A GPU embedding audit of all 1,796 translation–commentary pairs provides a corpus-scale review queue that correlates with independently flagged contested readings. We state precisely what is and is not verified: Lean checks the internal coherence of the formalization against the commentary-derived contract; the contract–commentary link remains human-auditable by construction, through verbatim citations. All artifacts — corpus, contracts, kernel, proofs, edition — are public and reproducible.

1 Introduction

The *Vākyapadīya* (“Treatise on Sentence and Word,” c. 5th century CE) is the foundational text of Indian philosophy of language. Its opening verse already illustrates the problem this paper addresses:

*anādinidhanaṃ brahma śabdatattvaṃ yadākṣaram /
vivartate ’rthabhāvena prakriyā jagato yataḥ //* (VP 1.1)

A translator must decide whether *śabdatattva* (“word-essence”) is a *property of language* or *the Absolute itself*; whether *vivartate* means the world is a *transformation* of Brahman (*pariṇāma*) or its *appearance without transformation* (*vivarta*); and whether *akṣara* means “imperishable,” “phoneme,” or — as the grammarian tradition insists — deliberately both. Each choice yields a grammatical English sentence; the choices differ philosophically as much as materialism differs from idealism. What disambiguates them is not syntax but the commentarial tradition: the vast apparatus of *vr̥tti* and *ṭikā* that fixes, verse by verse, which reading the text licenses [Subramania Iyer, 1965, Houben, 2016].

Translation of such texts has therefore been an art form in which the constraints, though real, live only in the philologist’s head. This paper makes them mechanical. Our claim is not that a theorem prover can read Sanskrit; it is that once a scholar’s interpretive commitments are stated as data, a proof assistant can *hold the translation to them* — exhaustively, incrementally, and publicly.

Contributions.

1. **Commentary-driven translation validation (C-TV).** We adapt translation validation from compiler verification [Pnueli et al., 1998, Leroy, 2009, Tristan and Leroy, 2008] to natural language: the commentary is compiled into per-verse *semantic contracts* (§4), and a translation’s reading is verified against the contract, not against the bare source string. To our knowledge this is the first application of an interactive theorem prover to translation adequacy in linguistics, and the first mechanized treatment of any Sanskrit philosophical text.¹
2. **A verified Brahmakāṇḍa.** All 144 verse units of Book I carry compiled adequacy theorems, and 350 plausible mistranslations are *compiled counterexamples* — refuted by theorem, not by footnote (§5). The full corpus of 1,796 verses with original translation and commentary is released as a checksummed, schema-validated digital edition.
3. **An anti-fabrication architecture for agentic formalization.** The contracts were authored by an LLM-agent swarm, and every layer is gated: axioms must cite the commentary *verbatim*; rejected readings must *provably* fail; entity sorts must be globally consistent or adjudicated as polysemy in a justified registry (§6, §7). We report what the gates caught — including fabricated citations in quota-degraded agent output — as evidence that the architecture, not agent goodwill, carries the epistemic load.
4. **A corpus-scale semantic audit.** A GPU embedding pass over all 1,796 translation–commentary pairs yields a drift-ranked review queue that correlates with the edition’s independently flagged *contested* readings (§8).

2 Background and Related Work

Bhartr̥hari and the sphoṭa doctrine. The *Vākyapadīya* comprises three books (Brahmakāṇḍa, Vākyakāṇḍa, Padakāṇḍa). Book I develops śabdādvaita — the thesis that the Absolute is of the nature of the word — and the sphoṭa doctrine: the meaning-bearing word is an undivided whole, merely *manifested* by the temporal sequence of sounds [Coward, 1980, Biardeau, 1964, Staal, 1969].

¹Prior mechanizations of Indian logic formalize the logical *systems* (§2); none, to our knowledge, verifies *translations of texts*.

Both doctrines generate exactly the translation hazards our system checks: sort errors (demoting the Absolute to a property), relation errors (rendering manifestation as transformation), and homonym errors (collapsing the technical senses of akṣara, sphoṭa, dhvani).

Computational Sanskrit. Pāṇinian grammar has long been recognized as a formal generative system [Kiparsky, 2009, Cardona, 1976, Scharf, 2009], and a mature computational ecosystem exists for segmentation, morphology and generation [Goyal et al., 2012, Hellwig and Nehrdich, 2018, Ambuda Project, 2024], with TEI-encoded editions in SARIT [SARIT Project, 2024]. This work is orthogonal and complementary: we verify *semantics against commentary*, not surface analysis.

Formalizing Indian logic. Navya-Nyāya has been studied logically since Ingalls [1951] and Matilal [1968]; recently Panday and Ghosh [2026] encoded its core constructs (relational nesting, delimiters, typed absence) in cubical type theory, Sathish [2026] trained LLMs on its six-phase epistemic method, and Diehl [2024] mechanized the Buddhist catuṣkoṭi in Lean. These formalize the *logics*; none verifies translations of a text. Our kernel borrows Navya-Nyāya’s structural insights (§5) in plain Lean 4 [de Moura and Ullrich, 2021], without cubical machinery.

Translation validation and autoformalization. Translation validation checks each compiler run *a posteriori* instead of verifying the compiler [Pnueli et al., 1998, Tristan and Leroy, 2008]. We transplant the pattern: the human translator is the untrusted optimizer; the contract is the semantic reference; Lean is the validator. On the authoring side, LLM autoformalization [Wu et al., 2022, Pathak, 2024] supplies the untrusted draft whose output the kernel gates — our loop is an instance of the generate-and-check paradigm with an unusually strict checker.

3 The Corpus

The edition is built from two sources: (i) the Devanagari mūla kārīkās scraped from a public edition (wisdomlib.org), and (ii) an original English translation and analytical commentary of every verse, written from the Sanskrit for this project. Alignment is checked, never assumed: the 1,796 commentary units (144 / 426 / 1,226 per book) are the canonical keys; every unit must match a mūla entry by exact label or be quarantined with a reason. The build is test-driven (12 corpus gates), and the tests forced three philologically real discoveries: the *contested* field is a prose note explaining each disputed reading (286 verses), not a boolean; verse markers legitimately embed Latin half-verse letters; and the source edition assigns the label 2.127cd to two different pages — a defect the commentary itself documents, which the loader merges and flags rather than silently overwriting. The resulting JSONL corpus carries per-record provenance and a manifest with a SHA-256 checksum.

4 Semantic Contracts

Definition 1 (Semantic contract). *A contract for verse v is a JSON document with (i) entities: named terms with ontological sorts drawn from {absolute, power, manifestation, linguisticItem, property, cognition}; (ii) axioms and denials: identity, relation and predication claims over those entities, each carrying a citation that must occur verbatim in the verse’s commentary; (iii) an accepted reading: the claims the translation asserts, each citing the translation verbatim; and (iv)*

Table 1: The Vākya-Vallārī corpus and the verified subset (Brahmakāṇḍa complete).

	Book I	Book II	Book III	Total
Verse units (canonical)	144	426	1,226	1,796
Contested readings (prose notes)	56	167	63	286
Semantic contracts	144	—	—	144
Adequacy theorems (Lean)	144	—	—	144
Refutation theorems (Lean)	350	—	—	350

rejected readings: *plausible mistranslations rendered as claim sets, each annotated with the failure mode it must exhibit* — *contradicted* (asserts a denial) or *unlicensed* (asserts what no axiom entails).

The verbatim-citation rule is the anti-fabrication gate: an axiom that cannot point to the commentary text that licenses it is rejected mechanically, before any proof is attempted. Across the 144 contracts there are 772 entity declarations, 610 axioms, 178 denials, 449 accepted-reading claims and 350 rejected readings. The validator further checks sort membership, ASCII identifier hygiene (a Lean requirement discovered the hard way, §6), entailment of the accepted reading, and provable failure of every rejected reading — the last via the same decision procedure the Lean kernel uses, so that Python-side acceptance predicts Lean-side theoremhood.

5 The Lean Kernel

The kernel (VakyaVallari, Lean 4.32, no external dependencies) is deliberately small. Entities are sort-tagged terms; claims are inductive data; a contract is axioms plus denials; and adequacy is *decidable*, so every verse-level theorem closes by **decide** — the proof term is the computation.

```

inductive Sorta | absolute | power | manifestation
                  | linguisticItem | property | cognition
structure Entity where name : String; sort : Sorta
-- paramparā: a relation can itself be a relatum, to any depth
inductive Node | ent : Entity → Node
                | rel : String → Node → Node → Node
structure Abhava where pratiyogin : Node; adhikarana : Node
structure Contract where axioms : List Claim; denials : List Claim
def Adequate (c : Contract) (r : Reading) : Prop :=
  r.claims.all (entails c.axioms) ∧ ¬ r.claims.any (denied c.denials)

```

Listing 1: Core of the adequacy kernel (abridged).

Three Navya-Nyāya-inspired choices matter. *Sorted identity*: claims of identity are stateable across sorts but decidable false when sorts differ, so a mistranslation that demotes the Absolute to a property fails by *sort error* — the C-TV failure mode as a computation. *Nested relations*: Node makes relations first-class relata (paramparā-sambandha), so “Time is the instrumental cause of the manifestation-relation” is representable without ad hoc reification. *Typed absence*: Abhava carries its counterpositive and locus as data, and the kernel proves Raghunātha’s involution (the absence of an absence computes to the presence) as a definitional fact.

Each contract compiles to a Lean module in which the accepted reading’s adequacy is a theorem and every rejected reading’s failure is a theorem. Verse 1.1’s module ends, in full:

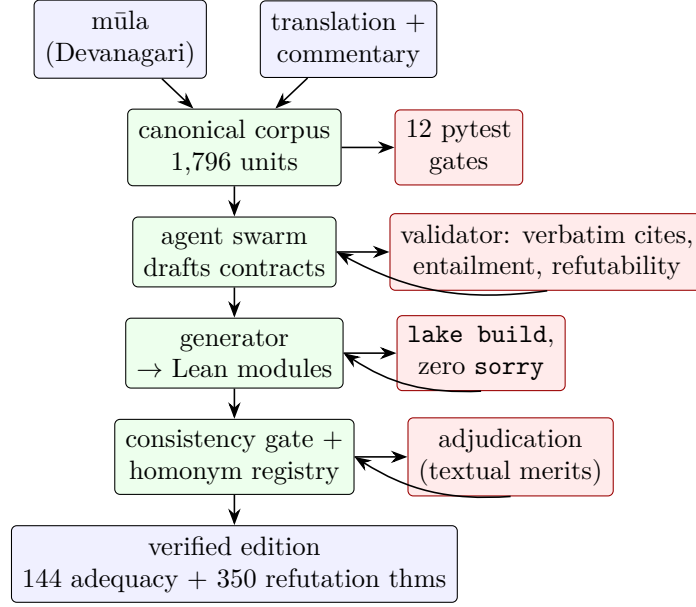


Figure 1: The C-TV pipeline. Every layer has a mechanical gate (red); failures feed back as repair work, and the append-only research ledger records each draft, rejection, repair and acceptance.

```

theorem accepted_adequate : contract.Adequate accepted := by decide
-- "The imperishable linguistic structure underlying words..."
theorem naive_linguistic_structure_inadequate :
  ¬ contract.Adequate naive_linguistic_structure := by decide
theorem naive_linguistic_structure_sort_error :
  sabdatattva_naive ≠ sabdatattva := by decide
-- "Brahman transforms itself into the world of objects."
theorem parinama_transformation_inadequate :
  ¬ contract.Adequate parinama_transformation := by decide

```

Listing 2: Generated theorems for VP 1.1 (excerpt).

The build gate is binary and global: `lake build` must succeed with zero `sorry` across all 144 modules — 549 theorems at present. A counterexample that stops compiling is a regression, caught in CI.

Honesty boundary. Lean verifies the *internal coherence of the formalization*: that the accepted reading satisfies the contract and rivals do not. Whether the contract is faithful to Bhartṛhari is not a theorem and cannot be one; it is a human-auditable question, and the system is built so that the audit is cheap — every axiom links to the exact commentary sentence it derives from. We regard this boundary not as a limitation to apologize for but as the correct division of labor between prover and philologist.

6 The Agentic Formalization Loop

Contracts were authored by an LLM-agent swarm — one agent per verse, each given the verse record, the schema, an exemplar, and the validator as its exit gate — orchestrated in rounds of

Table 2: Errors caught mechanically during formalization. “Lazy counterexample” = a rejected reading whose claims were secretly entailed; “fabricated citation” = an axiom cite absent from the commentary.

Failure class	Count	Caught by
Lazy counterexamples	3	Lean kernel, then hardened validator
Fabricated / paraphrased citations	5	verbatim-cite gate
Non-ASCII Lean identifiers	2 contracts	Lean parser, then validator
Cross-contract sort drift	89 conflicts	consistency gate
Diacritic name splits (sphoṭa/sphota)	2 episodes	consistency gate

9–30 agents with an adjudication pass between rounds and an append-only ledger (96 events) recording every draft, rejection, repair and acceptance. The division of labor is the point: agents propose; gates dispose. Four episodes illustrate what the gates caught.

(i) *Lazy counterexamples*. Early agents produced rejected readings that merely restated axioms — theorems of inadequacy would have been false. The Lean build failed; the validator was hardened to require provable failure; three contracts were repaired.

(ii) *Fabricated citations under degradation*. When an API session limit killed 20 of 30 agents mid-round, 9 left unreviewed draft contracts. Five of the nine failed the verbatim-citation gate — paraphrased or invented cites. This is the anti-fabrication architecture functioning as designed: degraded generation was caught by the gate, quarantined, and re-authored cleanly.

(iii) *Identifier hygiene*. Lean’s parser rejected IAST diacritics in entity identifiers that Python’s `isidentifier()` accepts; the validator now enforces ASCII identifiers, and the incident is a small argument for keeping the end-to-end kernel gate even when a faster mirror exists.

(iv) *Sort drift*. Independent agents assigned different sorts to the same term. Across five adjudication rounds the consistency gate surfaced 89 conflicts; adjudication (below) resolved each as either a correction or genuine polysemy.

7 Ontological Consistency and the Homonym Registry

The corpus-level gate requires that a named entity carry one sort everywhere — unless the exact partition of (verse, sort) uses is registered as polysemy, with a gloss and a one-sentence textual justification per sense. The registry now holds 41 terms with 91 senses, and it is where the method earns its keep philologically: the gate cannot tell drift from doctrine, so every split is a documented editorial ruling.

Three registered splits are load-bearing. *akṣara*: the imperishable Absolute in 1.9 versus phoneme in 1.18–22 — the pun the commentary calls “the argument in miniature” of verse 1.1, which a same-sort rule would have flattened. *śabda*: linguisticItem canonically, but the Absolute in the fire-in-wood verse (1.46), where the inner word is śabdatattva itself. *sphoṭa*: linguisticItem in Bhartṛhari’s own doctrine, but verses 1.102–1.106 *report a rival phonetician terminology* in which sphoṭa names the first-produced acoustic sound — a distinct sense (manifestation) covering exactly the doxographic span, keeping the report from contaminating the doctrine.

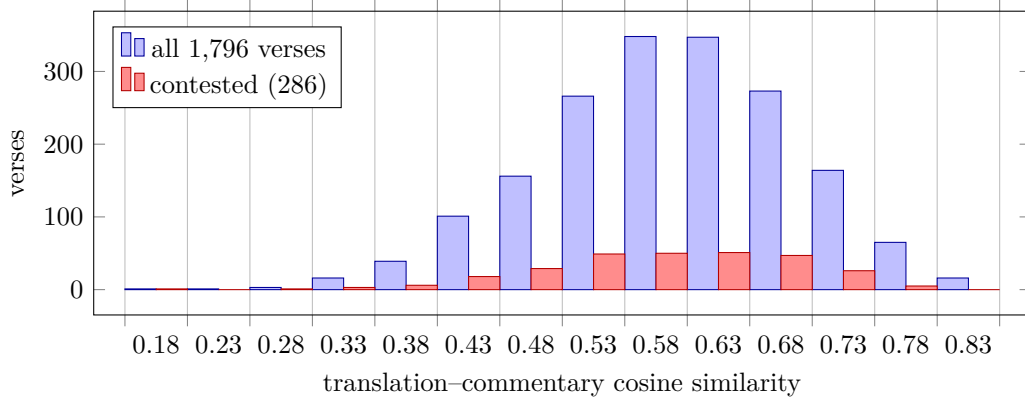


Figure 2: GPU semantic audit: distribution of translation-commentary embedding similarity over the full corpus, with the contested subset overlaid. Contested verses shift left (means 0.578 vs. 0.593); the lowest tail is the review queue.

Table 3: Verification results for the complete Brahmakāṇḍa.

Verse units formalized & verified	144 / 144
Entity declarations	772
Contract axioms / denials	610 / 178
Accepted-reading claims (all proved adequate)	449
Rejected readings, contradicted (denial asserted)	153
Rejected readings, unlicensed (no axiom entails)	197
Lean theorems (all by decide , zero sorry)	549
Homonym registry (terms / senses)	41 / 91
Python gates (pytest)	180
Ledger events (append-only)	96

8 Corpus-Scale Semantic Audit

Formal verification covers Book I; the remaining 1,652 verses need a prioritized path. We embed every verse’s translation and commentary (all-mpnet-base-v2 [Reimers and Gurevych, 2019], mean-pooled, on an NVIDIA GB10) and rank verses by cosine similarity. The distribution (Fig. 2) has mean 0.591; verses independently flagged *contested* score lower on average (0.578 vs. 0.593), and are enriched in the lowest tail (10 of the lowest 50, vs. a 15.9% base rate). The signal is modest and we use it only as a review queue: low similarity flags verses where translation and commentary may have drifted apart, for formalization or editorial attention first.

9 Results

Table 3 summarizes the verified state; CI reproduces it from a clean checkout (corpus gates, **lake** build, zero-**sorry** sweep). Beyond the counts, the qualitative yield is a machine-checked map of *how Book I can be mistranslated*: each of the 350 refutations is a specific, named failure — pariṇāma for vivarta (1.1), grammar demoted from discipline to convention (1.11–1.14), sphoṭa temporalized (1.75–1.77), inference granted autonomy the text denies it (1.30–1.42), the corrupt

form elevated to equal standing with the correct (1.147–1.155) — together a negative image of the doctrine, in theorem form.

The edition is deployed as a static site (per-verse pages with Devanagari, IAST, translation, commentary, contested notes, and per-verse proof status) with the repository, contracts, kernel and ledger public.

10 Discussion

What mechanization buys. Three things, in our experience of building this. *Regression*: interpretive commitments, once contracted, are protected — an edit to verse 1.46’s treatment of the inner word that contradicts 1.1’s identity axioms breaks the build. *Explicitness*: the contract format forced decisions (is *veda* here the corpus or the eternal seed-order?) that prose translation lets one defer; the homonym registry is a by-product catalogue of exactly those decisions. *Adversarial coverage*: requiring every contract to carry provably failing rival readings institutionalizes the question “what would it mean to get this verse wrong?” — a question translators ask irregularly and reviewers cannot ask at scale.

Limitations. The entailment relation is deliberately shallow (closure of literal claims under identity symmetry), so adequacy is membership-like rather than inferential; a richer relation (e.g. transitive sambandha chains, or defeasible generics) would strengthen the theorems at the cost of decidability discipline. Contracts formalize the *formalizable core* of a verse, not its full rhetorical content. The six-sort ontology is coarse — persons, for instance, are shoehorned (the child of 1.121/1.151 as epistemic subject). And the whole edifice inherits the interpretive standpoint of one commentary; contracts keyed to rival commentaries (Vṛṣabha, Puṇyārāja) would turn inter-commentarial disagreement itself into diffable, checkable data — to us the most interesting extension.

On agents. The swarm wrote in hours what would take a solo formalizer months, and its failures were institutionally useful: every class of agent error that occurred is now a mechanical gate. We suggest this as a general pattern for LLM-assisted scholarship: design so that the interesting question is never “did the model hallucinate?” but “did the artifact pass the gate?”

11 Conclusion

We introduced commentary-driven translation validation and delivered, for the first time to our knowledge, a machine-verified translation layer for a classical Sanskrit philosophical text: the complete Brahmakāṇḍa of the *Vākyapadīya*, with 144 adequacy theorems and 350 compiled refutations over a checksummed 1,796-verse edition, produced by a gated agentic loop whose every acceptance is replayable from an append-only ledger. The method treats the prover not as a reader of Sanskrit but as a keeper of commitments — and Bhartṛhari, for whom the sentence’s meaning is an undivided whole only manifested by its parts, would perhaps have appreciated a verifier that refuses to let the parts of a translation contradict the whole of its commentary. Books II and III (1,652 verses), richer entailment, and multi-commentary contracts are the open vine.

Availability. Code, corpus, contracts, kernel, proofs, ledger and the living edition: <https://github.com/SharathSPHD/vakya-vallari>; edition mirrors at <https://vakya-vallari.vercel>.

app and <https://huggingface.co/spaces/qbz506/vakya-vallari>.

References

- Ambuda Project. Vidyut: A high-performance Sanskrit toolkit. <https://github.com/ambuda-org/vidyut>, 2024.
- Madeleine Biardeau. *Théorie de la connaissance et philosophie de la parole dans le brahmanisme classique*. Mouton, Paris, 1964.
- George Cardona. *Pāṇini: A Survey of Research*. Mouton, The Hague, 1976.
- Harold G. Coward. *The Sphoṭa Theory of Language: A Philosophical Analysis*. Motilal Banarsidass, Delhi, 1980.
- Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.
- Stephen Diehl. Four-fold logic with Lean. https://www.stephendiehl.com/posts/buddhist_logic_lean/, 2024.
- Pawan Goyal, Gérard Huet, Amba Kulkarni, Peter Scharf, and Ralph Bunker. A distributed platform for Sanskrit processing. In *Proceedings of COLING 2012*, pages 1011–1028, 2012.
- Oliver Hellwig and Sebastian Nehrlich. Sanskrit word segmentation using character-level recurrent and convolutional neural networks. *Proceedings of EMNLP 2018*, pages 2754–2763, 2018.
- Jan E. M. Houben. Bhartṛhari. In Edward N. Zalta, editor, *The Stanford Encyclopedia of Philosophy*. 2016.
- Daniel H. H. Ingalls. *Materials for the Study of Navya-Nyāya Logic*. Harvard University Press, Cambridge, MA, 1951.
- Paul Kiparsky. On the architecture of Pāṇini’s grammar. *Sanskrit Computational Linguistics, LNCS*, 5402:33–94, 2009.
- Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7): 107–115, 2009.
- Bimal Krishna Matilal. *The Navya-Nyāya Doctrine of Negation*. Harvard University Press, Cambridge, MA, 1968.
- Ekansh Panday and Sujata Ghosh. Cubical type theoretic Navya-Nyāya. *arXiv preprint arXiv:2605.12548*, 2026.
- Shashank Pathak. GFLean: An autoformalisation framework for Lean via GF. *arXiv preprint arXiv:2404.01234*, 2024.
- Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 1384 of *LNCS*, pages 151–166. Springer, 1998.

- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- SARIT Project. SARIT: Search and retrieval of indic texts. <https://sarit.indology.info/>, 2024. TEI-encoded electronic editions of Sanskrit texts.
- Sharath Sathish. Pramana: Teaching large language models six-phase Navya-Nyāya epistemic reasoning. *arXiv preprint arXiv:2604.04937*, 2026.
- Peter M. Scharf. Modeling Pāṇinian grammar. *Sanskrit Computational Linguistics, LNCS*, 5402: 95–126, 2009.
- J. F. Staal. Sanskrit philosophy of language. *Current Trends in Linguistics*, 5:499–531, 1969.
- K. A. Subramania Iyer. *The Vākyapadīya of Bhartr̥hari with the Vṛtti: Chapter I*. Deccan College, Poona, 1965.
- Jean-Baptiste Tristan and Xavier Leroy. Formal verification of translation validators: A case study on instruction scheduling optimizations. In *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 17–27, 2008.
- Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. Autoformalization with large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 32353–32368, 2022.